

KEYSIGHT

TECHNOLOGIES

CloudLens Ansible

AWS Customer Runbook

Automated sensor deployment for Linux and Windows EC2 at scale

Version	v1.0.0
Date	June 2026

Keysight Technologies | Network Visibility Solutions

Table of Contents

- 1. Executive Summary.....
- 2. Solution Overview.....
- 3. Choosing Your Deployment Path.....
- 4. Prerequisites Checklist.....
- 5. Deployment, Step by Step.....
 - 5.1 Tier 1: One-Click Launch Stack (CloudFormation).....
 - 5.2 Tier 2: AWS CloudShell.....
 - 5.3 Tier 3: Docker (laptop or CI/CD).....
- 6. Supported EC2 Scenarios.....
- 7. Verification Checklist.....
- 8. Scaling Guide.....
- 9. Troubleshooting Reference.....
- Appendix A: customer_input.yaml Schema.....
- Appendix B: Bulk Tag Script.....
- Appendix C: Quick Links.....

1. Executive Summary

CloudLens Ansible for AWS is a fully automated kit that deploys Keysight CloudLens sensors to every tagged EC2 instance in an AWS account. It works on Linux and Windows, from a single instance to 5,000+, without any manual per-host steps.

It is built for customer DevOps and network teams who already run Ansible and need network visibility instrumented quickly, and for Keysight Sales Engineers who need a repeatable, demoable proof-of-value asset. The Ansible layer connects over SSH, AWS Systems Manager (SSM), or WinRM, so it works even in accounts where one of those is locked down.

Why it matters

- Eliminates the per-instance SSH / RDP grind. Tag an instance with `cloudlens=yes` and the kit handles the rest.
- Three entry points (CloudFormation Launch Stack, CloudShell curl, or Docker), so customers run it wherever they already work.
- Same playbook portable to Azure and GCP, so multi-cloud teams keep one workflow.

Proof points

Scenario	Result	Time
847-instance fleet (mixed Ubuntu/RHEL/Windows)	100% sensor registration	45 min
Ubuntu 22.04 (SSH)	Sensor running	4 min
Windows Server 2022 (SSM)	Sensor running	6 min
vController registration	All sensors Connected	<1 min

Numbers from production deployments and the deployment runbook. Reference account region: us-east-1.

2. Solution Overview

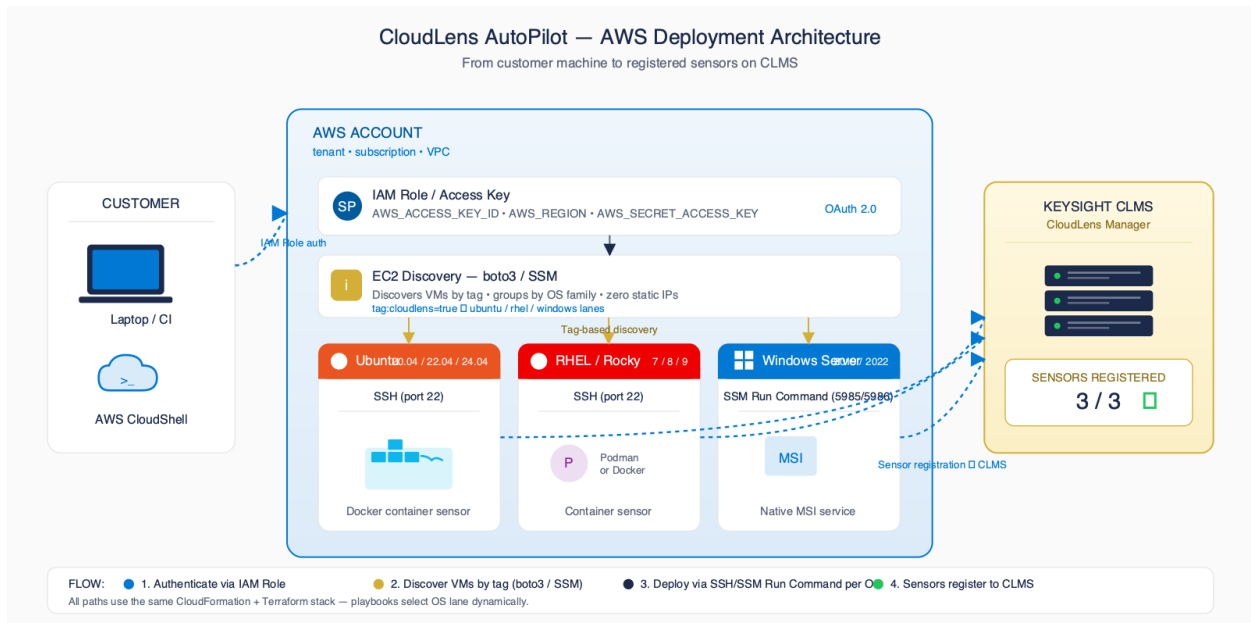


Figure 1: End-to-end architecture (control point -> AWS inventory -> OS lanes -> vController)

A single Ansible control point authenticates to AWS (SSO profile, access keys, or a CloudShell session), queries EC2 for every instance matching the customer's tag filter, and routes each host to the OS-specific playbook lane: Ubuntu, RHEL, or Windows.

Linux hosts run the CloudLens agent in a Docker or Podman container. Windows hosts run the CloudLens Windows sensor as a native service. Every sensor self-registers with CloudLens vController (formerly CLMS) on first start using the project key supplied in `customer_input.yaml`. For the packet path, KVO orchestrates AWS VPC Traffic Mirroring into collector SVMs and a vPB. No per-instance UI steps, no static inventory files to maintain.

3. Choosing Your Deployment Path

All three paths run the same Ansible engine. Same playbooks, same automation. Pick the entry point that matches how your team works.

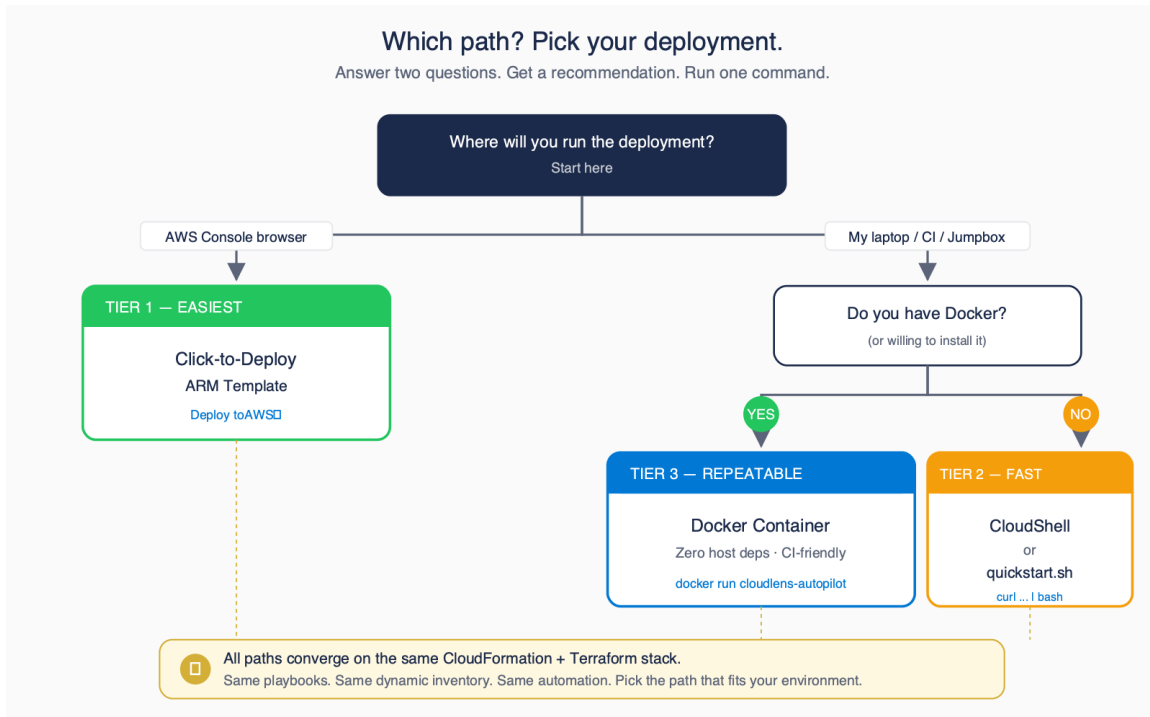


Figure 2: Decision tree (pick the entry point that matches your environment)

Tier	Best For	Tools Needed	Effort
Tier 1: Launch Stack	Customers who live in the AWS Console and want zero local setup	Web browser only	Lowest
Tier 2: CloudShell	Customers already authenticated to AWS in the browser	AWS CloudShell (pre-authenticated)	Low
Tier 3: Docker	Repeatable runs from a laptop or CI/CD pipeline	Docker, AWS credentials, customer_input.yaml	Medium

4. Prerequisites Checklist (Printable)

Confirm every item below before kicking off a deployment. Tick each box as you go. The kit will not magically fix a missing prerequisite.

- ☐ AWS account with rights to describe EC2 and (for Tier 1) create the CloudFormation stack.
- ☐ Subscribed to the three Keysight Marketplace AMIs (Vision One, CloudLens Manager, Virtual Packet Broker), one time per account.
- ☐ vController (CLMS) reachable from the target instance subnets on TCP/443.
- ☐ Project Key obtained from the vController UI (Settings > Projects > API Keys).
- ☐ Target instances tagged with `cloudlens=yes`, `os=ubuntu|rhel|windows`, `env=prod|dev|qa`.
- ☐ EC2 key pair (Linux SSH) or SSM Agent + IAM role (SSM mode) ready.

Tip

Tags are how the dynamic inventory discovers hosts. If an instance is not tagged, the kit cannot see it. That is the single most common support ticket. Use the bulk-tag script in Appendix B to tag an entire region in one shot.

5. Deployment, Step by Step

5.1 Tier 1: One-Click Launch Stack (CloudFormation)

Best when you want zero local tools. The Launch Stack button opens the CloudFormation quick-create console with the stack template pre-loaded from this repository. It deploys vController, KVO (optional), and vPB (optional) from the Marketplace AMIs, then you run the sensor playbook.

1. Open the README or the public site and click the 'Launch Stack' button.
2. Sign in to the AWS Console when prompted; the CloudFormation quick-create form opens with the template URL filled in.
3. Fill the parameters: EC2 key pair, admin ingress CIDR, and which components to deploy (KVO and vPB default to yes).
4. Acknowledge IAM capabilities and click Create Stack. About 5 minutes later the stack reaches CREATE_COMPLETE.
5. Read the stack Outputs for the vController, KVO, and vPB URLs and IPs.
6. Complete the vController EULA + first login, create a project, and copy the project key, then run the sensor playbook (Tier 2 or Tier 3).

Expected outcome

Stack reaches CREATE_COMPLETE in about 5 minutes. vController, KVO, and vPB come up on their own Elastic IPs. Instance types are fixed to the size each AMI is qualified on (vController and vPB t3.xlarge, KVO c5.2xlarge).

5.2 Tier 2: AWS CloudShell

Best when the customer is already logged into AWS in a browser tab. CloudShell is pre-authenticated, so there is no access key to manage and no local install.

7. Open AWS CloudShell from the console toolbar (region us-east-1).
8. Run the quickstart bootstrap:
9. Answer the wizard prompts: vController IP, project key, tag filter, connection mode (SSH / SSM / WinRM).
10. Confirm sensors in vController > Sensors (about 8 min for a small fleet).

```
curl -sSL https://raw.githubusercontent.com/Keysight-Tech/cloudlens-ansible-aws/main/quickstart.sh | bash
```

Expected outcome

Wizard discovers tagged instances, auto-tunes Ansible forks, deploys sensors, and prints a success summary. All state lives in your CloudShell home directory, with nothing installed locally.

5.3 Tier 3: Docker (laptop or CI/CD)

Best for repeatable runs from a developer laptop, a CI pipeline, or any container host. The image is pinned and hermetic, so you get the same result on macOS, Windows, Linux, GitHub Actions, GitLab CI, and Jenkins.

11. Copy `customer_input.yaml.example` to `customer_input.yaml` and fill in your vController IP, project key, region, and tag filter.
12. Provide AWS credentials to the container (mount `~/.aws` read-only, or pass `AWS_ACCESS_KEY_ID` / `AWS_SECRET_ACCESS_KEY` as env vars).
13. For Linux SSH targets, mount your EC2 key pair path read-only.
14. Run the container:

```
docker run --rm -it \  
-v $(pwd)/customer_input.yaml:/work/customer_input.yaml \  
-v $HOME/.aws:/root/.aws:ro \  
-v $HOME/.ssh:/root/.ssh:ro \  
-e AWS_PROFILE -e AWS_DEFAULT_REGION=us-east-1 \  
ghcr.io/keysight-tech/cloudlens-ansible-aws:latest
```

Expected outcome

Container exits 0 with a 'sensors deployed: N/N' summary. Pin the image tag in CI to get hermetic, reproducible runs.

6. Supported EC2 Scenarios

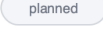
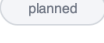
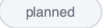
Supported VM Scenarios				
Operating system x network topology — verified deployment paths				
OS / Distribution	Public IP direct SSH/SSM Run Command	Private + Jumpbox	AWS Bastion	CloudShell browser-based
 Ubuntu 20.04 · 22.04 · 24.04				
 RHEL · Rocky · Alma 7 · 8 · 9				
 CentOS 7 · 8				
 Windows Server 2019 · 2022		 planned	 planned	
 Supported & verified  planned Planned — not yet tested Honest status. We mark only what we have validated end-to-end.				

Figure 3: EC2 compatibility matrix (OS x topology x connection method)

OS / Topology	Public IP + SSH	Private + Bastion	SSM (no inbound)	CloudShell
Ubuntu 20.04 / 22.04 / 24.04	Supported	Supported	Supported	Supported
RHEL 7 / 8 / 9, Amazon Linux	Supported	Supported	Supported	Supported
Rocky / AlmaLinux	Supported	Supported	Supported	Supported
Windows Server 2019 / 2022	Supported (WinRM)	Supported (SSM)	Supported	Supported

7. Verification Checklist (Printable)

After every deployment, walk this list. If any item fails, jump to section 9 (Troubleshooting Reference).

- ☐ All sensors visible in vController > Sensors page
- ☐ Filter by custom_tags (for example Customer=Acme) returns the expected instances
- ☐ KVO > Inventory > Devices shows vController and vPB as CONNECTED
- ☐ KVO > Cloud Fabric > Cloud Configs shows the AWS config COMMITTED
- ☐ Container logs show no errors (Linux): docker logs cloudlens-agent
- ☐ CloudLens service running (Windows): Get-Service CloudLensAgent

8. Scaling Guide

Fleet Size	Parallelism	Sharded?	Approx Time
1 to 50	50 concurrent	No	5 to 10 min
51 to 200	100 concurrent	No	10 to 20 min
201 to 800	200 concurrent	No	15 to 30 min
801 to 2,000	400 concurrent	Yes (auto)	30 to 60 min
2,001 to 5,000	800 concurrent	Yes	1 to 2 hr
10,000+	2,500+ concurrent	Yes	4+ hr

The kit auto-tunes the Ansible fork count based on the number of instances the dynamic inventory discovers. Above 2,000 instances it also auto-shards: the inventory is split into batches and the playbook runs them in controlled waves. This keeps memory and AWS API throttling under control while still finishing in a reasonable window for tens of thousands of hosts. See docs/SCALING.md in the repository for KVO infrastructure sizing and VPC Traffic Mirroring limits.

9. Troubleshooting Reference

Organised by symptom area. The full reference lives at docs/TROUBLESHOOTING.md and docs/OPERATIONS.md in the repository.

Inventory

Symptom	Likely Cause	Fix
Inventory finds 0 instances	Tags missing on instances	<code>aws ec2 create-tags --resources <id> --tags Key=cloudlens,Value=yes</code>
Subset of instances missing	Tag value mismatch (case-sensitive)	Use exact lowercase values: ubuntu, rhel, windows, prod

SSH (Linux)

Symptom	Likely Cause	Fix
Permission denied (publickey)	Wrong key path or SSH user	Set <code>aws.ssh_key_path</code> and the correct <code>ssh_user_*</code> in <code>customer_input.yaml</code>
Connection timeout	Security group blocks port 22 from control point	Switch to bastion or SSM connection mode

SSM (Linux and Windows)

Symptom	Likely Cause	Fix
SSM shows 0 target instances	Missing IAM role or SSM Agent stopped	Attach AmazonSSMManagedInstanceCore; confirm <code>aws ssm describe-instance-information</code>
session-manager-plugin not found	AWS CLI SSM plugin not installed on control node	Install session-manager-plugin, or switch <code>linux_connection</code> to ssh

Registration with vController

Symptom	Likely Cause	Fix
Sensor not in vController UI	Wrong project key	Check Settings > Projects > API Keys; update <code>customer_input.yaml</code>
Sensor offline after restart	vController IP unreachable from instance subnet	Open egress from the subnet to vController on TCP/443

Appendix A: customer_input.yaml Schema

Full annotated example. Copy customer_input.yaml.example from the repository to customer_input.yaml and fill in values. Never commit customer_input.yaml to git. Credentials belong in your AWS profile or env vars.

```
# === AWS Environment ===
aws:
  profile: "default"
  regions:
    - us-east-1

  # Tag selector: instances are matched by these tags
  tag_filters:
    cloudlens: "yes"
    env:       "prod"

  windows_connection: "ssm"      # ssm | winrm
  linux_connection:   "ssh"      # ssh | ssm
  ssh_key_path:       "~/aws/your-key.pem"
  ssh_user_ubuntu:    "ubuntu"
  ssh_user_rhel:      "ec2-user"

# === CloudLens Configuration ===
cloudlens:
  manager_ip_or_fqdn: "vcontroller.customer.example.com"
  project_key:        "REPLACE_WITH_PROJECT_KEY"
  custom_tags:        "Env=AWS Region=us-east-1 Customer=Acme"
  registry_type:      "insecure"  # or "secure"
  linux_runtime:      "auto"      # auto | docker | podman

# Optional scoping: empty lists = all VPCs / subnets in region
vpc_ids:  []
subnet_ids: []

# === Deployment Behavior ===
deploy:
  forks: 0          # 0 = auto-tune
  timeout_seconds: 60
  reinstall_if_unhealthy: true
  shard_size: 500   # for >2000 instances
```

Appendix B: Bulk Tag Script

Apply the three required tags to instances in a region, filtered by platform. Run these from AWS CloudShell or any shell with the AWS CLI configured.

Ubuntu / Linux instances

```
aws ec2 describe-instances --region us-east-1 \
  --filters Name=instance-state-name,Values=running \
    Name=platform-details,Values="Linux/UNIX" \
  --query "Reservations[].Instances[].InstanceId" --output text \
| xargs -n1 -I {} aws ec2 create-tags --resources {} \
  --tags Key=cloudlens,Value=yes Key=os,Value=ubuntu Key=env,Value=prod
```

Windows instances

```
aws ec2 describe-instances --region us-east-1 \
  --filters Name=instance-state-name,Values=running \
    Name=platform,Values=windows \
  --query "Reservations[].Instances[].InstanceId" --output text \
| xargs -n1 -I {} aws ec2 create-tags --resources {} \
  --tags Key=cloudlens,Value=yes Key=os,Value=windows Key=env,Value=prod
```

Appendix C: Quick Links

Resource	Location
GitHub repository	https://github.com/Keysight-Tech/cloudlens-ansible-aws
Public site	https://keysight-tech.github.io/cloudlens-ansible-aws/
Deployment guide	docs/DEPLOYMENT_GUIDE.md (in the repo)
Operations guide	docs/OPERATIONS.md (in the repo)
Scaling guide	docs/SCALING.md (in the repo)
Troubleshooting guide	docs/TROUBLESHOOTING.md (in the repo)
Customer email templates	docs/CUSTOMER_EMAIL.md (in the repo)
GitHub Issues (bug reports / feature requests)	https://github.com/Keysight-Tech/cloudlens-ansible-aws/issues

Support email subject (for the Keysight account team): 'CloudLens Ansible AWS: <customer name>, <brief issue>'.